



TEST SUITE MINIMISATION IN LASSO USING STIMULUS-RESPONSE MATRICES

Bachelor Thesis

submitted: 01.December 2025

by: Laith Philipp Sandouk

`laith.sandouk@students.uni-mannheim.de`

born May 15th, 2005

in Mannheim

Student ID Number: 1993811

Chair of Software Engineering
School of Business Informatics and Mathematics
University of Mannheim
D – 68159 Mannheim
Phone: +49 621-181-3912, Fax +49 621-181-3909
Internet: <http://swt.informatik.uni-mannheim.de>

Abstract

Large-scale industrial software systems face significant challenges in executing test suites efficiently. In a commercial SAP environment, Bach et al. report test suites comprising more than 130,000 tests requiring hundreds of hours of sequential runtime [3]. Such execution times render continuous testing computationally infeasible and economically prohibitive. The study further revealed extensive redundancy within these test suites, implying that substantial execution cost yields only marginal additional coverage benefit. However, this is not an isolated case, test-suite redundancy is a widespread and well-documented issue in industrial software development. While this computational burden cannot be eliminated entirely, its impact can be substantially reduced through test-suite minimisation, which systematically removes redundant tests with respect to defined adequacy criteria such as statement coverage, branch coverage, or mutation score [27]. Test-suite minimisation is particularly critical for regression testing, where previously passing tests are re-executed after code changes to detect newly introduced defects. However, naive reduction approaches risk compromising fault-detection effectiveness, as tests with unique defect-revealing capabilities may be inadvertently excluded.

This thesis investigates test-suite minimisation in the context of the *Large-Scale Software Observatory* (LASSO) [16], a research platform for automated selection, execution, and comparative analysis of software systems. LASSO integrates open-source repositories and combines static and dynamic analyses within a unified framework designed for language independence. Its current implementation focuses on Java and Python systems and enables empirically grounded studies of *Big Code* ecosystems (large, diverse corpora of source code enriched with meta-data such as bug-fix histories, version control traces, and execution outcomes [2]).

Our approach leverages *Stimulus–Response Matrices* (SRMs) [16], a two-dimensional representation mapping test stimuli to observed execution outcomes. We enrich SRM entries with per-test coverage information and utilise the SRM’s existing

mutation data, enabling minimisation algorithms to operate on fine-grained behavioural information rather than language-specific code structures.

Through a targeted literature survey encompassing greedy heuristics, exact optimisation, search-based, and data-driven minimisation strategies, we comparatively assessed candidate algorithms along six established criteria: SRM compatibility, coverage retention, reduction effectiveness, mutation-coverage preservation, computational scalability, and implementation feasibility. The Mutation-Aware Enhanced Harrold–Gupta–Soffa (MA-EHGS) algorithm emerged as the most promising candidate. MA-EHGS extends the empirically validated Enhanced HGS baseline [10] by integrating mutation weighting to balance coverage preservation and fault-detection power [14].

We successfully integrated MA-EHGS into LASSO and validated the implementation using real-world SRMs. The achieved reduction levels fall within the ranges reported for HGS-based and mutation-aware greedy techniques in prior empirical studies [10, 14]. Since MA-EHGS follows the same matrix-driven selection structure as other greedy test-suite minimisation algorithms, it retains the practical scalability widely observed for greedy heuristics in regression testing [27, 20].

Contents

Abstract	ii
List of Figures	viii
List of Tables	ix
1. Introduction	1
1.1. Background and Motivation	1
1.2. LASSO Context	2
1.3. Research Objectives and Questions	2
1.4. Implementation Strategy	3
1.5. Thesis Structure	3
2. Fundamentals	4
2.1. Test Coverage and Mutation Score	4
2.1.1. Coverage Metrics	4
2.1.2. Mutation Score	5
2.2. Stimulus–Response Matrices	5
2.2.1. Sequence and Actuation Sheets	5
2.2.2. From Sequence Sheets to SRMs	6
2.3. The Test-Suite Minimisation Problem	6
2.3.1. Concept and Objectives	6
2.3.2. Computational Complexity and Trade-offs	7
2.3.3. Minimisation within LASSO	8
2.3.4. Algorithmic Categories	8
3. Literature Review	9
3.1. Greedy Heuristics	9
3.1.1. Classical Greedy	9
3.1.2. Harrold–Gupta–Soffa	10

3.1.3. Delayed Greedy	10
3.1.4. Enhanced HGS	10
3.1.5. Greedy Heuristics with Overlap Awareness	11
3.1.6. Mutation-Aware Enhanced HGS	11
3.2. Exact Optimisation Approaches	12
3.2.1. ILP, SAT, and Set-Cover Formulations	12
3.2.2. NEMO and Nonlinear Formulations	12
3.2.3. REDUNET	12
3.3. Search-Based and Metaheuristic Approaches	13
3.3.1. Genetic Algorithms	13
3.3.2. Other Metaheuristic Approaches	13
3.4. Data-Driven and Similarity-Based Approaches	13
3.4.1. Similarity-Based and Time-Constrained Methods	13
3.5. Summary	14
4. Evaluation Criteria and Scoring Framework	15
4.1. Evaluation Criteria (C1–C6)	15
4.1.1. Mandatory Precondition	15
4.1.2. Performance Criteria	15
4.1.3. Justification of Criteria	16
4.2. Ranking and Scoring Model	17
5. Comparative Evaluation and Selection	18
5.1. Criterion C1: SRM Compatibility	18
5.2. Criterion C2: Coverage Preservation	19
5.3. Criterion C3: Reduction Effectiveness	20
5.4. Criterion C4: Mutation Score Preservation	20
5.5. Criterion C5: Computational Scalability	21
5.6. Criterion C6: Implementation Feasibility	21
5.7. Selection Decision	22
6. Implementation	24
6.1. Architecture Overview	24
6.2. Data Extraction and Representation	25
6.3. Algorithmic Realisation	25
6.4. Integration Into LASSO	26

Contents	vi
6.5. Validation	26
7. Discussion	27
7.1. General Advantages and Disadvantages of Test Suite Minimisation	27
7.1.1. Practical Performance Benefits	27
7.1.2. The Absence of a Perfect Solution	28
7.1.3. The Coverage Limitation	29
7.2. Thesis-Specific Considerations for MA-EHGS in LASSO	29
7.2.1. Advantages of the Approach	29
7.2.2. Limitations of the Implementation	30
7.2.3. Implications for LASSO Platform Evolution	31
7.3. Future Work	32
7.3.1. Multi-Language Support	32
7.3.2. Multisystem Minimisation	32
7.3.3. Integration Testing Validation	32
7.3.4. Alpha–Beta Benchmarking	32
7.3.5. Weak Mutation Support	33
7.3.6. LLM-Assisted Gap Analysis	33
8. Conclusion	34
8.1. Summary	34
8.2. Key Contributions	34
8.3. Future Work	35
Bibliography	vii
Appendix	xi
Detailed Evaluation Tables for Comparative Analysis	xii
A.1. Complete Criterion Scores	xii
A.1.1. C1: SRM Compatibility	xii
A.1.2. C2: Coverage Preservation	xiv
A.1.3. C3: Reduction Effectiveness	xv
A.1.4. C4: Mutation Score Preservation	xvii
A.1.5. C5: Computational Scalability	xix
A.1.6. C6: Implementation Feasibility	xx

B.2. Complete Multi-Criteria Scoring Table	xxii
C.3. MA-EHGS Selection Justification	xxii
C.3.1. Mutation-Awareness Advantage (C4: Key Differentiator) .	xxii
C.3.2. Multi-Criteria Optimality	xxiii
C.3.3. Alternatives and Trade-offs	xxiii
C.3.4. Practical Advantages for LASSO	xxiv
D.4. Reference Summary	xxiv
Mutation-Aware Enhanced HGS Pseudocode	xxv
A.1. Extended Benchmarks and Reproducibility	xxvii
B.2. SRM Coverage Extraction Specification	xxviii
B.2.1. SRM Structure and Storage	xxviii
B.2.2. Per-Test Coverage Collection	xxviii
B.2.3. Data Collector Query Strategy	xxix
B.2.4. Critical Implementation Details	xxx
B.2.5. Output Format	xxx
C.3. Licensing Header Template	xxxi

List of Figures

2.1. Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).	6
2.2. Example SRM fragment showing per-test outcomes across different system variants. Behavioural discrepancies can be used to compute mutation scores and coverage information.	6
2.3. The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.	8

List of Tables

1.	C1: SRM Compatibility — full evaluation.	xii
2.	C2: Coverage Preservation — score breakdown.	xiv
3.	C3: Reduction Effectiveness — empirical percentages and scoring.	xv
4.	C4: Mutation Score Preservation — critical gap analysis.	xvii
5.	C5: Computational Scalability — complexity analysis for $n >$ 10,000 tests.	xix
6.	C6: Implementation Feasibility — infrastructure requirements. . .	xx
7.	Complete multi-criteria scores: all 13 algorithms ranked (C1 = pass/fail; C2–C6 = 0–5; total = 0–25).	xxii
1.	Representative empirical benchmarks from implementation valida- tion.	xxvii

1. Introduction

Software testing ensures that program modifications do not introduce defects. As systems evolve, test suites grow and increase execution cost and maintenance effort. Modern suites often combine human-written, automatically generated, and in recent years more frequently AI-generated tests. Tools such as Randoop [22] and EvoSuite [8] can produce thousands of tests, and large industrial suites may require days or weeks to execute [24]. Although such suites achieve high coverage and mutation scores, they frequently contain redundant tests whose requirements are subsumed by others, resulting in prolonged execution, increased costs, and elevated maintenance overhead.

Test-suite minimisation addresses this issue by systematically removing redundant tests while preserving fault-detection capability [27]. Techniques range from *coverage-preserving* approaches, which maintain all coverage requirements, to *coverage-relaxing* approaches, which accept limited coverage loss for stronger reduction. While minimisation can significantly reduce computational demand and feedback latency, overly aggressive reduction risks discarding tests with unique defect-revealing behaviour. Effective minimisation therefore requires balancing reduction efficiency with fault-detection retention.

1.1. Background and Motivation

Testing remains a major cost driver in software engineering. Classical studies estimate that verification and validation account for up to half of total development effort [4]. The cost pressure persists in modern continuous-integration environments, where automated or AI-generated suites may contain tens of thousands of tests [24, 3]. Executing all tests after each change is often infeasible, motivating techniques that reduce suite size while retaining quality. Test suite minimisation has been exploring this issue for decades, with numerous algorithms proposed to

identify and eliminate redundant tests [27], by leveraging coverage and quite recently mutation information as a proxy for fault-detection capability [14]. Details on coverage and mutation testing are provided in Chapter 2.

1.2. LASSO Context

The *Large-Scale Software Observatory* (LASSO) [16] is a research platform for scalable analysis of large software corpora (*Big Code*). LASSO automates the extraction of static properties and dynamic execution behaviour from open-source systems and enables reproducible pipelines through the *LASSO Scripting Language* (LSL). This makes it a central infrastructure for empirical software engineering and for evaluating emerging techniques such as AI-assisted code generation.

LASSO represents system behaviour through *Stimulus–Response Matrices* (SRMs) [16], where rows denote test stimuli, columns denote systems, and cells contain structured response objects. SRMs allow language-independent extraction of coverage metrics, mutation scores, and behavioural characteristics, enabling systematic comparison of functionally equivalent implementations. Section 2.2 provides formal definitions.

Despite supporting large-scale behavioural and coverage analysis, LASSO currently lacks integrated test-suite minimisation. Redundant test executions accumulate significant computational overhead across the platform. Integrating minimisation would reduce this overhead while preserving the behavioural coverage required for reliable system analysis.

1.3. Research Objectives and Questions

This thesis investigates how test-suite minimisation can be integrated into LASSO using its SRM abstraction. The study is guided by:

1. **RQ1:** Which minimisation techniques demonstrate the highest practical effectiveness in the literature for the LASSO platform?
2. **RQ2:** Which evaluation criteria best suit minimisation within LASSO’s SRM-based environment?

3. **RQ3:** Which approach best balances reduction effectiveness, coverage preservation, mutation preservation and scalability?
4. **RQ4:** How can the selected approach be integrated into LASSO while maintaining language independence?

1.4. Implementation Strategy

This thesis performs minimisation directly on SRMs rather than relying on external tools, which typically require source-code access or language-specific instrumentation. SRMs already encapsulate all data required for behavioural minimisation—test identifiers, execution results, coverage information, and mutation data—allowing language-independent and scalable processing while ensuring seamless integration into LASSO’s analysis pipeline.

1.5. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 introduces coverage, mutation testing, SRMs, and formally defines the minimisation problem.

Chapter 3 surveys greedy, exact, search-based, and data-driven minimisation techniques.

Chapter 4 defines evaluation criteria (C1–C6) and the scoring framework.

Chapter 5 applies the framework to select the Mutation-Aware Enhanced HGS approach.

Chapter 6 describes the implementation and integration into LASSO.

Chapter 7 discusses limitations, trade-offs, and deployment considerations.

Chapter 8 concludes and outlines future work.

2. Fundamentals

This chapter establishes the theoretical foundation for this work. It introduces the core concepts of test coverage and mutation testing, explains the *Stimulus–Response Matrix* (SRM) representation used within LASSO, and formalises the test-suite minimisation problem, including trade-offs between coverage preservation and reduction strength.

2.1. Test Coverage and Mutation Score

Test coverage quantifies the proportion of program elements exercised by a test suite [28]. Coverage metrics indicate structural thoroughness but provide limited insight into fault-detection capability.

2.1.1. Coverage Metrics

Coverage criteria define program ”parts” at different granularity levels [28]. While numerous coverage types exist (e.g., instruction coverage, path coverage, class coverage), this work focuses on the three metrics most commonly reported in test-suite minimisation literature [27, 11, 14, 20]:

- **Line Coverage:** proportion of source lines executed.
- **Branch Coverage:** proportion of conditional branches whose true/false outcomes are both executed.
- **Method Coverage:** proportion of methods invoked at least once.

These three metrics capture execution at different granularities and are widely supported by standard coverage tools such as JaCoCo. Combining multiple metrics provides a more complete assessment of test quality, as high line coverage with low branch coverage may miss conditional faults.

2.1.2. Mutation Score

Mutation testing measures test effectiveness by introducing small syntactic changes (*mutants*) into the program and observing whether tests detect the injected faults [15]. The *mutation score* is defined as:

$$\text{Mutation Score} = \frac{\text{Number of mutants killed}}{\text{Total number of mutants}} \times 100 \, \%.$$

A mutant is *killed* if the test causes observable behavioural divergence from the original program.

Mutation Types. Two principal mutation styles are distinguished [21]:

1. **Weak Mutation:** detects state deviations immediately after a mutated statement executes.
2. **Strong Mutation:** detects externally visible behavioural differences at program output.

Weak mutation captures local sensitivity to code changes. Strong mutation quantifies global behavioural divergence. Mutation scores thereby approximate a suite’s fault-detection capability.

2.2. Stimulus–Response Matrices

Stimulus–Response Matrices (SRMs) constitute LASSO’s central data structure for *Test-Driven Software Experimentation* [16]. They provide a unified, language-independent format for recording and comparing run-time behaviour under controlled stimuli.

2.2.1. Sequence and Actuation Sheets

Sequence Sheets describe ordered stimuli applied to systems under test. Each row represents one method invocation with inputs and expected outputs. **Actuation Sheets** record the corresponding observed outputs and other run-time data during execution.

Stimulus (Sequence Sheet)				Observed Behaviour (Actuation Sheet)			
Out	Operation	Service	Input	Out	Operation	Service	Input
create	create	'Stack					
–	push	A1	42	<instance>	create	'Stack	
42	pop	A1		true	push	A1	42
				42	pop	A1	

Figure 2.1.: Relationship between a sequence sheet (stimulus) and its corresponding actuation sheet (observed behaviour).

2.2.2. From Sequence Sheets to SRMs

LASSO generalises stimulus-to-actuation mappings into multidimensional *Stimulus–Response Matrices* (SRMs):

$$M = [M_{ij}], \quad M_{ij} = \text{Actuation}(t_i, v_j),$$

where each row t_i represents a stimulus (test), each column v_j a system variant, and each cell M_{ij} stores execution outcomes and associated analysis data such as coverage and mutation attributes.

	System 1	System 2	System 3	System 4
Test 1	pass	fail	pass	fail
Test 2	pass	pass	pass	pass
Test 3	pass	fail	fail	pass

Figure 2.2.: Example SRM fragment showing per-test outcomes across different system variants. Behavioural discrepancies can be used to compute mutation scores and coverage information.

SRMs unify functional results, coverage data, and mutation outcomes in a single matrix, enabling language-independent behavioural analysis.

2.3. The Test-Suite Minimisation Problem

2.3.1. Concept and Objectives

Test-suite minimisation seeks to remove redundant tests from an existing suite while maintaining sufficient coverage and fault-detection capability [27]. Given a

suite $T = \{t_1, \dots, t_n\}$ and a set of coverage requirements $R = \{r_1, \dots, r_m\}$, each test t_i covers a subset $C(t_i) \subseteq R$. The objective is to find a smaller subset $S^* \subseteq T$ that provides comparable or identical coverage.

Two major formulations exist:

- **Coverage-preserving minimisation:** guarantees full coverage, $C(S^*) = C(T)$. It ensures no requirement is lost but often limits reduction potential.
- **Coverage-relaxing minimisation:** allows limited loss of coverage, $C(S^*) \subset C(T)$, to achieve greater reduction. This variant trades fault-detection certainty for smaller suite size.

Both forms address the same optimisation tension: *minimise suite size while retaining sufficient adequacy*. The general optimisation can be expressed as

$$S^* = \arg \min_{S \subseteq T} |S| \quad \text{subject to} \quad |C(S)| \geq \gamma \cdot |C(T)|,$$

where $0 < \gamma \leq 1$ controls the acceptable coverage retention threshold ($\gamma = 1$ for coverage-preserving, $\gamma < 1$ for coverage-relaxing).

2.3.2. Computational Complexity and Trade-offs

The coverage-preserving variant corresponds to the *minimum set-cover problem*, which is NP-complete [27]. No polynomial-time algorithm guarantees optimality for arbitrary inputs, and practical implementations rely on heuristics or approximations [11, 6]. Some greedy heuristics achieve linear complexity $O(nm)$ in practice, balancing scalability and quality.

Coverage-relaxing algorithms may yield stronger reductions but risk discarding unique defect-revealing tests, potentially lowering mutation scores or missing rare execution paths. Hence, every minimisation approach must balance the trade-off between computational efficiency, reduction rate, and fault-detection retention.

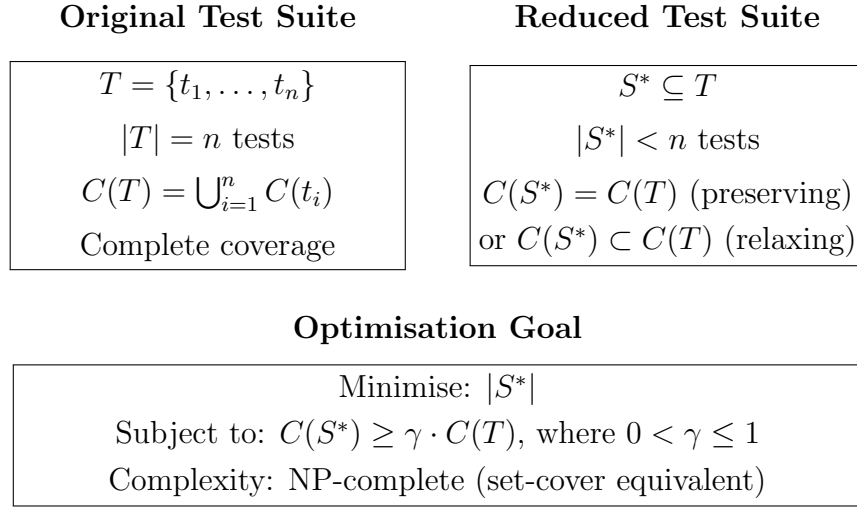


Figure 2.3.: The test-suite minimisation problem formulated as an optimisation balancing suite size and coverage retention.

2.3.3. Minimisation within LASSO

In LASSO, each test corresponds to one SRM row. Coverage and mutation data stored in SRM cells provide the requirement mappings $C(t_i)$. Minimisation thus operates directly on SRMs by selecting a subset of rows S^* that preserves coverage and mutation adequacy across all variants:

$$C(S^*)_{\text{SRM}} \geq \gamma \cdot C(T)_{\text{SRM}}.$$

This SRM-based formulation supports both coverage-preserving and coverage-relaxing strategies while remaining language-independent, enabling scalable experimentation across diverse software systems.

2.3.4. Algorithmic Categories

Following established classifications [27, 23, 3], algorithms group into four categories: **greedy heuristics** (classical greedy, HGS [11]), **exact optimisation** (ILP, constraint-satisfaction [18]), **search-based methods** (genetic algorithms [27]), **data-driven methods** (clustering, overlap-aware [3]). Detailed explanation in Chapter 3.

3. Literature Review

This chapter surveys test-suite minimisation algorithms across four categories: greedy heuristics, exact optimisation, search-based approaches and data-driven techniques, focusing on methods that operate on or can be expressed via a requirements matrix (SRM).

3.1. Greedy Heuristics

Greedy heuristics are among the most widely studied minimisation techniques [27, 20]. They are typically formulated as instances of the minimal hitting-set (set-cover) problem: given tests and requirements (e.g. statements, branches, mutants), a greedy algorithm iteratively selects tests that cover the most yet-uncovered elements [6, 27]. Because minimal hitting-set is NP-complete [9] and test-suite minimisation is commonly formulated as a hitting-set or set-cover problem [13, 20], greedy heuristics serve as practical approximations rather than exact solvers.

3.1.1. Classical Greedy

The classical greedy heuristic repeatedly selects the test covering the most uncovered requirements. Chvátal proved a logarithmic approximation bound [6], and subsequent work treats this heuristic as a baseline for regression-test minimisation [24, 27].

Noemmer and Haas note that greedy variants are “the test suite minimization algorithms most commonly found in research” [20]. In their Java study, both classical greedy and HGS (Section 3.1.2) achieve substantial reduction while preserving statement coverage; mutation testing shows that, under adequate reduction, mutation scores remain close to those of the original suite, whereas more aggressive reduction can incur relative mutation-score losses of about 3.5–21%, depending on project and threshold [20]. Jehan and Wotawa report similar findings

for JavaScript projects, with roughly 70% reduction and only minor differences between greedy variants under adequate settings [14].

3.1.2. Harrold–Gupta–Soffa

Harrold, Gupta and Soffa (HGS) propose an early coverage-driven heuristic [11]. Their method partitions tests by covered requirements and selects them in phases, prioritising requirements with few covering tests while avoiding redundancy.

Harrold et al. give a worst-case time bound of $O(n(n + m)MAXCARD)$, where n denotes the number of associated testing sets, m the number of test cases, and $MAXCARD$ the maximum number of requirements covered by any single test [11]. HGS remains a standard greedy baseline [27, 5, 20]. Chen and Lau introduced GE and GRE, finding no universally superior greedy technique [5]. Jehan and Wotawa report that HGS achieves around 70% reduction when all mutation requirements must be met, with only small differences in fault detection across greedy variants [14].

3.1.3. Delayed Greedy

Tallam and Gupta introduced delayed greedy to mitigate a weakness of classical greedy and HGS: selecting large tests too early may render smaller tests redundant [25]. Their approach uses concept analysis to remove subsumed tests before greedy selection.

They show that delayed greedy runs in polynomial time and often yields equal or smaller suites than classical greedy or HGS [25]. Jehan and Wotawa confirm similar fault-detection behaviour across greedy variants under adequate reduction, with delayed greedy typically producing the smallest suites at a higher runtime [14].

3.1.4. Enhanced HGS

Gladston and Ramesh refine HGS and evaluate several variants on Java programs [10]. *Enhanced HGS* selects tests with unique coverage first, then chooses remaining tests by residual coverage.

On the FIRST RUN benchmark, they report suite-size reductions (SSR) of 73.51% (GRE), 69.01% (HGS), 71.83% (Non-Redundant HGS) and 73.66% (Enhanced HGS) [10]. Enhanced HGS typically matches or slightly improves upon HGS and GRE, though the evaluation remains coverage-based.

3.1.5. Greedy Heuristics with Overlap Awareness

Overlap-aware greedy heuristics penalise tests with highly redundant coverage. Bach et al. apply such approaches in a SAP environment to maximise coverage under tight time budgets rather than obtain minimal suites [3]. Similarity-based and clustering methods identify redundant tests and prioritise those providing new coverage.

These methods improve achieved coverage within fixed budgets but are aimed at prioritisation rather than exact minimisation [3].

3.1.6. Mutation-Aware Enhanced HGS

Mutation-Aware Enhanced HGS (MA-EHGS), proposed in this thesis, extends Enhanced HGS by incorporating mutation information, motivated by evidence that coverage-only minimisation may reduce mutation scores [20, 14].

Requirements-Matrix Formulation

MA-EHGS operates on a binary matrix $M \in \{0, 1\}^{n \times m}$ where rows represent tests and columns represent requirements (structural coverage and mutants). Mutants are encoded as additional columns, treating both requirement types uniformly.

Two-Phase Selection with Mutation Weighting

MA-EHGS follows Enhanced HGS's structure:

Phase 1: Select tests that uniquely cover any requirement.

Phase 2: Select tests using the weighted score

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}_i| + \beta \cdot |\text{newMutants}_i|,$$

where $\alpha, \beta \in [0, 1]$ balance structural and mutation requirements.

Complexity and Performance

MA-EHGS identifies essential tests and scores candidates via uncovered requirements, retaining the scalability of other greedy heuristics [27, 20, 14] while adding mutation awareness.

3.2. Exact Optimisation Approaches

Exact approaches model minimisation as ILP-, SAT- or CSP-based optimisation problems [27, 23]. These formulations ensure optimality for the encoded objective but inherit NP-hardness because hitting-set is NP-complete [9] and commonly used to represent test-suite minimisation [24, 20].

3.2.1. ILP, SAT, and Set-Cover Formulations

ILP formulations express requirements as constraints and tests as decision variables, minimising the number (or cost) of selected tests [27, 7]. SAT and CSP encodings follow the same principle [12]. Though exponential in the worst case, ILP/SAT can solve small- to medium-sized instances and serve as optimality baselines [27, 23].

3.2.2. NEMO and Nonlinear Formulations

Lin et al. suggest that linear models miss test interdependencies [17]. NEMO uses a multi-criteria nonlinear integer program, adding objectives such as cost and fault detection. It can substantially improve fault-detection capability (up to 164%) compared with linear ILP, but at far higher computational cost.

3.2.3. REDUNET

REDUNET combines set-cover reasoning with optimisation on control-flow graphs (CFGs) [18]. It uses CFG structure with ILP reasoning to detect redundant tests. Because it relies on CFGs, it is not applicable in SRM-only settings.

3.3. Search-Based and Metaheuristic Approaches

Search-based methods treat minimisation as an optimisation problem using metaheuristics such as genetic algorithms or swarm intelligence [27, 23]. These allow multi-objective optimisation but typically incur higher computational cost and require parameter tuning.

3.3.1. Genetic Algorithms

Genetic Algorithms (GAs) evolve populations of candidate subsets through selection, crossover and mutation [27]. They can produce highly reduced suites but require substantial tuning and are more expensive than greedy heuristics [27, 23]. Many GA-based approaches still optimise primarily for coverage rather than mutation [20].

3.3.2. Other Metaheuristic Approaches

Ahuja and Bhatia evaluate PSO and ABC algorithms, reporting strong cost reductions; GAs sometimes produce smaller suites depending on parameters and program characteristics [1]. Wei et al. apply mutation-aware many-objective evolutionary algorithms (NSGA-II, MOEA/D, IBEA, SPEA2-SDE) and show that including mutation score preserves fault detection while reducing cost [26].

3.4. Data-Driven and Similarity-Based Approaches

Data-driven approaches analyse coverage vectors to detect redundancy [27, 23].

3.4.1. Similarity-Based and Time-Constrained Methods

Similarity-based clustering and ranking methods group tests by coverage similarity [27, 23]. Bach et al. apply such techniques in a SAP environment under strict time budgets [3]. Penalising overlap improves coverage within time limits but targets prioritisation rather than strict minimisation [3, 23].

3.5. Summary

The surveyed literature suggests:

- **Greedy variants as practical baselines.** Classical greedy, HGS and delayed greedy achieve substantial reduction while preserving coverage, with small differences in mutation preservation under adequate settings [27, 20, 14]. Enhanced HGS offers modest improvements [10].
- **Coverage–mutation gap.** Coverage-only minimisation can reduce mutation scores [20]. Mutation-aware greedy and evolutionary methods preserve fault detection more effectively [14, 26].
- **Exact methods vs. scalability.** ILP, SAT and nonlinear models yield strong optimality baselines but become expensive as problem size grows, reflecting hitting-set hardness [9, 27, 17, 20].
- **Search-based and data-driven trade-offs.** Metaheuristics and similarity-based methods support flexible objectives but require tuning and incur higher cost than greedy heuristics [23]. Overlap-aware methods are primarily used for prioritisation [3].
- **Motivation for mutation-aware greedy minimisation.** Mutation-guided greedy and evolutionary work shows that incorporating mutation information helps preserve fault detection [14, 26]. MA-EHGS extends a validated greedy heuristic with mutation awareness while maintaining SRM-based scalability.

4. Evaluation Criteria and Scoring Framework

Building upon the fundamentals established in the previous chapter, this chapter defines six evaluation criteria (C1–C6)—one mandatory precondition and five performance dimensions—together with a scoring model for systematic algorithm comparison. These criteria operationalise the research questions and provide the foundation for evaluating different minimisation algorithms in the next chapter.

4.1. Evaluation Criteria (C1–C6)

4.1.1. Mandatory Precondition

C1: SRM Compatibility (Mandatory) The algorithm must operate directly on *Stimulus–Response Matrix* (SRM) representations without requiring control-flow graphs, syntax trees, or language-specific structural analysis [16]. Algorithms failing C1 receive a score of zero and are excluded. This precondition guarantees language-independent execution across heterogeneous software systems.

4.1.2. Performance Criteria

C2: Coverage Retention Measures how much of the original structural coverage (instruction, line, branch) is retained after minimisation [28]. High retention indicates that the reduced suite maintains structural adequacy.

C3: Reduction Effectiveness Quantifies the proportion of test cases eliminated while preserving required coverage [27]. A high reduction rate reflects strong elimination of redundant tests and tangible computational savings.

C4: Mutation-Coverage Preservation Assesses fault-detection capability via mutation testing [14, 20]. The mutation score—ratio of killed mutants to total

mutants—acts as a fault-based adequacy metric complementing structural coverage.

C5: Computational Scalability Examines algorithm performance on large SRMs as the number of tests and requirements increases. Methods with near-linear complexity and efficient runtime behaviour are favoured [16, 3].

C6: Implementation Feasibility Considers practical integration factors such as implementation effort, parameter tuning, dependency overhead, and compatibility with LASSO’s architecture [27]. Algorithms imposing heavy maintenance or configuration costs are penalised.

4.1.3. Justification of Criteria

The six criteria align directly with the research questions from Section 1.3, ensuring coherence between theoretical goals and empirical assessment.

C1 (SRM Compatibility) is mandatory because LASSO operates entirely on SRM abstractions [16]. Language independence requires that algorithms process abstract requirement matrices rather than language-specific artefacts.

C2–C4 (Quality Preservation) address adequacy and fault detection. **C2** ensures structural completeness (coverage retention). **C3** quantifies the degree of reduction. **C4** measures mutation-score retention, a strong empirical proxy for test effectiveness and fault detection capability [14]. Together, C2 and C4 provide comprehensive assurance that the minimised test suite maintains both structural coverage and fault-revealing capability.

C5 (Computational Scalability) ensures the algorithm remains feasible for large-scale use cases where SRMs may contain thousands of rows and columns [16, 3].

C6 (Implementation Feasibility) recognises the cost of practical adoption. Even theoretically optimal algorithms may become impractical if they require complex configuration or structural changes to the LASSO pipeline [27].

4.2. Ranking and Scoring Model

To enable systematic comparison, each algorithm is scored on a scale from 0 to 5 for each performance criterion (C2–C6). The scoring rubric is as follows:

- **Score 5:** Excellent performance; meets or exceeds best-known benchmarks in literature.
- **Score 4:** Good performance; achieves strong results with minor limitations.
- **Score 3:** Acceptable performance; meets basic requirements but with notable trade-offs.
- **Score 2:** Poor performance; significant shortcomings limit practical utility.
- **Score 1:** Very poor performance; fails to meet essential criteria.
- **Score 0:** Not applicable; fails mandatory precondition (C1).

After scoring, each algorithm's total score is computed as a sum with equal weighting across criteria except C1 (which is mandatory):

$$\text{Score}(A) = \begin{cases} 0, & \text{if } s_1(A) = 0 \text{ (fails C1),} \\ \sum_{i=2}^6 w_i \cdot s_i(A), & \text{otherwise.} \end{cases} \quad (4.1)$$

This scoring framework facilitates quantitative comparison across diverse algorithms, enabling informed selection based on empirical performance aligned with LASSO's requirements. The next chapter applies this framework to evaluate candidate minimisation techniques from the literature, ultimately selecting the most suitable approach for integration into LASSO.

5. Comparative Evaluation and Selection

This chapter evaluates candidate test-suite minimisation techniques against the criteria defined in Chapter 4 and justifies the selection of the Mutation-Aware Enhanced HGS (MA-EHGS) algorithm for integration into LASSO. The goal is not to re-explain algorithms—Chapter 3 provides that—but to determine which techniques offer the best trade-offs for LASSO’s Stimulus–Response Matrix (SRM) setting. Numerical scores and tables appear in Appendix 8.3; here we focus on qualitative conclusions.

The evaluation considers the following technique families:

- **Greedy heuristics:** classical greedy, HGS, Enhanced HGS, Delayed Greedy, GE/GRE, and mutation-based greedy variants.
- **Exact optimisation:** ILP, SAT/constraint encodings and nonlinear formulations such as NEMO.
- **Search-based approaches:** genetic algorithms and many-objective evolutionary algorithms (MOEAs).
- **Data-driven methods:** similarity- and overlap-based algorithms.
- **Hybrid structural techniques:** REDUNET, which combines set-cover reasoning with CFG analysis.

All methods are evaluated along criteria C1–C6: SRM compatibility, coverage preservation, reduction effectiveness, mutation-score preservation, scalability and implementation feasibility.

5.1. Criterion C1: SRM Compatibility

C1 requires algorithms to operate directly on SRMs without relying on program-structure analyses such as CFGs or ASTs. Most examined techniques satisfy this.

Classical greedy heuristics, HGS, Enhanced HGS and Delayed Greedy all operate on test–requirement incidence matrices [11, 5, 27, 20]. ILP/SAT formulations similarly encode set cover over tests and requirements [24, 27, 7]. NEMO adds multi-criteria optimisation but retains the same matrix representation [17]. Search-based techniques represent each candidate solution as a subset of tests defined over coverage and mutation vectors [27, 23, 26]. Similarity- and overlap-based approaches also rely on coverage vectors [27, 3].

By contrast, REDUNET depends on control-flow graphs and associated program structures [18], which LASSO’s SRM abstraction does not provide. It therefore violates C1.

Conclusion (C1). All algorithms except REDUNET satisfy C1. C1 thus serves mainly as an exclusion filter.

5.2. Criterion C2: Coverage Preservation

C2 requires that minimisation preserve structural coverage under adequate settings. Greedy heuristics (classical greedy, HGS, Enhanced HGS, Delayed Greedy, GE/GRE) all ensure coverage preservation by construction: they select tests until all required elements are covered [11, 5, 27, 20]. Noemmer and Haas and Jehan and Wotawa adopt this requirement explicitly in their evaluations [20, 14].

Exact ILP/SAT encodings guarantee coverage preservation whenever a feasible solution is found [7, 27]. NEMO also enforces coverage constraints while optimising additional criteria [17].

Search-based methods may either enforce coverage as a hard constraint or treat it as one objective among several [27, 23, 26]. Similarly, similarity-based approaches typically optimise coverage under time budgets rather than strict adequacy [3]. Only hard-constraint configurations satisfy LASSO’s C2.

Conclusion (C2). Greedy heuristics, ILP/SAT and NEMO satisfy C2 under adequate settings. Evolutionary and similarity-based methods can satisfy C2 but only when configured to enforce coverage strictly.

5.3. Criterion C3: Reduction Effectiveness

C3 evaluates how strongly an algorithm reduces the test suite while meeting C2.

Exact ILP and SAT encodings compute minimal hitting sets for the modelled constraints [24, 7]. NEMO extends this with multi-criteria optimisation and reports substantial improvements over linear ILP baselines [17].

Among heuristic methods, several studies show strong performance for greedy variants. Enhanced HGS and Non-Redundant HGS outperform HGS on their benchmarks, achieving average suite-size reductions around 72–74% compared with 69% for HGS [10]. Jehan and Wotawa report around 70% reduction for multiple greedy variants under adequate mutation requirements [14]. Delayed Greedy guarantees reductions no worse than classical greedy and often better in practice [25].

Search-based methods such as mutation-aware MOEAs also achieve substantial reductions [26], though cross-study comparisons can vary due to differing subjects and evaluation setups.

Conclusion (C3). ILP/SAT/NEMO provide optimal or near-optimal bounds but at higher cost. Among heuristics, Enhanced HGS, Delayed Greedy and related greedy variants consistently achieve strong reduction. MA-EHGS inherits this behaviour from Enhanced HGS.

5.4. Criterion C4: Mutation Score Preservation

C4 assesses how well minimisation preserves mutation score. Earlier studies focused mainly on structural coverage [27, 23], but recent work incorporates mutation explicitly.

Noemmer and Haas show that coverage-based minimisation can reduce mutation score when coverage thresholds drop below 100% [20]. Jehan and Wotawa demonstrate that greedy minimisation preserves mutation score when all mutants are required but may lose mutants when requirements are relaxed [14]. Wei et al. use mutation score as an optimisation objective in MOEAs and show that mutation-aware objectives preserve fault detection while reducing cost [26].

MA-EHGS extends Enhanced HGS by treating mutants as additional requirements and weighting them via parameter β . This provides explicit mutation-awareness while retaining a deterministic, SRM-based heuristic.

Conclusion (C4). Mutation-aware greedy and evolutionary methods provide the strongest mutation preservation. Purely coverage-based heuristics preserve mutation score under adequate settings but not under relaxed ones. MA-EHGS offers a balanced, SRM-compatible mechanism for incorporating mutation information.

5.5. Criterion C5: Computational Scalability

C5 examines algorithmic scalability as test-suite and requirement sizes grow. As test-suite minimisation is NP-complete [9, 24], exact methods face inherent limits, whereas heuristics sacrifice optimality for efficiency.

Greedy heuristics scale well and are widely used in practice [27, 20, 14]. Tallam and Gupta show that Delayed Greedy runs in polynomial time [25]. Gladston and Nehemiah report practical performance for Enhanced HGS on medium-sized suites [10]. Noemmer and Haas confirm scalability on real projects [20].

Exact ILP/SAT techniques solve small to medium instances but scale poorly as requirements increase [7, 27]. NEMO handles multi-criteria problems but remains expensive [17]. Search-based methods incur further cost due to population-based optimisation, requiring many evaluations per run [26]. Similarity-based methods scale well computationally but target prioritisation under time constraints [3].

MA-EHGS matches the scalability of Enhanced HGS, with mutation requirements simply adding more columns to the SRM.

Conclusion (C5). Greedy SRM-based heuristics (classical greedy, HGS, Enhanced HGS, Delayed Greedy, MA-EHGS) offer the best scalability. Exact and evolutionary methods incur substantially higher computational cost.

5.6. Criterion C6: Implementation Feasibility

C6 considers practical integration: dependencies, configuration load, determinism and compatibility with LASSO’s SRM architecture.

Greedy algorithms are simple to integrate, deterministic and require no external solvers [27, 20, 14]. Enhanced HGS and Delayed Greedy extend this with modest heuristic additions [25, 10].

ILP/SAT methods require solver dependencies [7, 27]. NEMO requires nonlinear optimisation infrastructure [17]. Search-based methods require evolutionary operators, objective modelling and parameter tuning [27, 23, 26], and introduce non-determinism. Similarity-based approaches rely on execution-time metadata and domain-specific similarity choices [3].

MA-EHGS remains deterministic, SRM-based and requires only two weighting parameters (α , β).

Conclusion (C6). SRM-based greedy heuristics require the least engineering effort and fit naturally into LASSO. Solver-based, evolutionary and similarity-based techniques involve significantly higher integration complexity.

5.7. Selection Decision

Across C1–C6, several algorithms perform well individually:

- **C2 (coverage):** classical greedy, HGS, Enhanced HGS, Delayed Greedy, ILP/SAT and NEMO [11, 24, 7, 10, 25, 27, 20].
- **C3 (reduction):** ILP/SAT/NEMO provide optimal reductions; Enhanced HGS and Delayed Greedy achieve strong heuristic reductions [25, 10, 20, 14].
- **C4 (mutation):** mutation-based greedy and mutation-guided MOEAs [14, 26].
- **C5–C6 (scalability/feasibility):** greedy matrix-based heuristics [27, 20, 14].

Combining the criteria, classical greedy, Enhanced HGS, Delayed Greedy, mutation-based greedy variants and MA-EHGS all score highly. The key differentiator for this thesis is C4: explicit mutation-awareness in an SRM-compatible, deterministic algorithm.

MA-EHGS is selected because it:

- operates directly on SRMs (C1),
- preserves structural coverage when $\alpha > 0$ (C2),
- inherits strong reduction behaviour from Enhanced HGS (C3),
- incorporates mutation awareness via weighted mutant requirements (C4),
- retains greedy scalability (C5),
- and integrates into LASSO without external dependencies (C6).

Other algorithms excel on individual criteria—e.g. ILP/SAT/NEMO on optimality or MOEAs on multi-objective flexibility—but at the cost of scalability or complexity. Purely coverage-based greedy heuristics perform well on C2, C3, C5 and C6 but lack mutation-awareness (C4).

Overall, MA-EHGS provides the most favourable balance of SRM compatibility, coverage preservation, reduction effectiveness, mutation-awareness, scalability and implementation feasibility for LASSO. The following chapter presents its implementation and empirical evaluation.

6. Implementation

This chapter presents the technical implementation of the Mutation-Aware Enhanced HGS minimiser within LASSO. The work was developed in a dedicated branch:

`<https://github.com/.../lasso-minimization>`

and the full reference implementation is available at:

`<https://github.com/.../MinimizationAlgorithm.java>`

The contribution of this work lies in four areas: reliable per-test coverage collection, mutation-aware requirement extraction, a unified requirements-matrix representation, and a scalable realisation of the Enhanced HGS algorithm extended with mutation weighting. All components integrate seamlessly into the existing Arena–Mutation–SRM workflow and remain fully language-independent.

6.1. Architecture Overview

The implementation adds a cohesive data-processing layer that retrieves execution data from SRMs, converts them into requirement mappings, and applies the minimiser before writing back a reduced SRM. Central to this design is a unification layer that merges structural coverage and mutation information into a single requirements matrix. This preserves LASSO’s language-agnostic design and enables consistent minimisation across different programming languages.

Three new components form the backbone of the implementation. The `SRMCoverageExtractor` collects per-test coverage from SRM sheets generated during Arena execution and standardises them using a language-neutral `COVERAGE.testName` prefix, replacing earlier tool-specific formats. The `MutationAnalyzer` reconstructs mutation kills from SRM outputs by comparing original and mutant variants cell-by-cell, resolving previous inaccuracies caused by single-cell comparisons or null handling. Finally, the `BitSetMatrix` provides a compact, high-performance representation

of all test–requirement relations, with structural elements and mutants encoded uniformly.

6.2. Data Extraction and Representation

Coverage is collected by executing JaCoCo instrumentation separately for each test, generating isolated coverage sheets per test. By adopting language-neutral identifiers, the design remains extensible to future Python or JavaScript coverage integrations.

Mutation information is reconstructed from SRM entries of type `value`. A mutant is considered killed if any output cell diverges from the original implementation:

$$\exists(x, y) : M_{i,\text{orig}}(x, y) \neq M_{i,m}(x, y).$$

This refined method yields a precise kill signal and corrects inconsistencies of earlier pipelines that observed only the first output cell.

Coverage and mutation elements are unified into the set

$$R = R_{\text{coverage}} \cup R_{\text{mutation}},$$

which forms the input to the minimisation algorithm.

6.3. Algorithmic Realisation

The minimiser implements the two-phase Enhanced HGS algorithm. Phase 1 selects all tests that uniquely satisfy a requirement, ensuring that both structural and mutation-specific elements with cardinality one are always preserved. Phase 2 applies a weighted greedy selection:

$$\text{score}(t_i) = \alpha \cdot |\text{newCoverage}(t_i)| + \beta \cdot |\text{newMutants}(t_i)|,$$

with default parameters $\alpha = 0.3$ and $\beta = 0.7$. BitSet operations keep runtime at $O(nm)$ and allow efficient experimentation with different scoring configurations.

6.4. Integration Into LASSO

The minimiser is exposed as the LSL action `TestSuiteMinimization`. It executes after both Arena and Mutation, consumes SRM data from Ignite, performs minimisation, and optionally replaces the SRM with a reduced version.

Listing 6.1 shows a representative configuration. Configurable parameters are shown in [blue color](#).

Listing 6.1: LSL configuration for the minimisation action

```
action(name: 'minimize', type: 'TestSuiteMinimization') {
  dependsOn 'mutate'
  includeAbstractions 'Base64'

  alpha = 0.3
  beta  = 0.7
  keepMutants = true
  keepOracle = false
}
```

The implementation required updating SRM extraction logic, correcting test-name normalisation, introducing language-agnostic sheet identifiers, and refining mutation comparison behaviour. These changes collectively make SRM-based minimisation correct, robust, and compatible with diverse software systems.

6.5. Validation

Validation confirmed that all structural and mutation requirements are preserved for $\alpha > 0$ and $\beta > 0$, and that the algorithm behaves deterministically under fixed configurations. Unit tests verify requirement mapping, kill-detection correctness, and algorithmic behaviour. Benchmarks (Appendix A.1) show consistent reductions of 46.9–55.6% with full retention of structural and mutation requirements.

7. Discussion

This chapter critically analyses test suite minimisation from two perspectives: general advantages and disadvantages inherent to the technique, followed by specific considerations for the Mutation-Aware Enhanced HGS implementation within LASSO’s architectural constraints.

7.1. General Advantages and Disadvantages of Test Suite Minimisation

Test suite minimisation represents a sophisticated optimisation challenge where no single approach universally excels. This section examines inherent trade-offs and fundamental constraints independent of specific algorithmic implementations.

7.1.1. Practical Performance Benefits

Empirical evidence from large-scale industrial environments underscores the significant performance gains achievable through test suite minimisation. Bach et al. [3] observed that over 90% of test cases in a major industrial project (SAP) were redundant in terms of line coverage, with full test execution requiring several tens of hours. Applying minimisation techniques in such contexts can reduce execution time from many hours to just a few hours—or even minutes—without compromising coverage quality.

These reductions yield immediate and measurable benefits. A 50–75% shrinkage in test suites translates directly into lower computational demands, reduced infrastructure expenditure, and substantial cost savings. Minimised test execution drastically decreases CPU hours, memory allocation, and cloud consumption, which in turn cuts operational costs and energy usage. In continuous integration pipelines running hundreds of builds per day, the cumulative effect can be dra-

matic: a 60% reduction across 500 daily builds eliminates roughly 300 redundant test executions per cycle [27], saving both time and significant financial resources.

Shorter test cycles also enhance developer productivity. Compressing test execution times from hours to minutes accelerates feedback loops, allowing defects to surface earlier and be resolved faster. The ability to detect critical failures almost immediately reshapes development practices, especially in agile environments that rely on frequent integration and rapid iteration.

7.1.2. The Absence of a Perfect Solution

Despite these compelling advantages, test suite minimisation remains an inherently complex optimisation problem with no universally optimal solution. The difficulty arises from conflicting objectives and problem-specific constraints that vary across application domains. Minimisation confronts multiple competing goals: maximising reduction while preserving coverage, maintaining fault-detection capability, ensuring computational feasibility, and adapting to diverse system characteristics. No technique satisfies all objectives simultaneously across all contexts.

Different application domains prioritise these objectives differently. Safety-critical systems demand absolute fault-detection preservation, favouring conservative approaches that sacrifice reduction potential. High-frequency continuous integration pipelines prioritise execution speed, accepting greater risk of fault-detection loss. Large-scale observatories require language-agnostic solutions operating on behavioural abstractions, excluding techniques dependent on source code analysis.

This multi-objective optimisation creates a complex solution space where trade-offs vary by context. Conservative approaches produce larger minimised suites with stronger guarantees. Aggressive approaches achieve greater reduction with weaker assurances. The optimal balance depends on deployment context, testing objectives, and acceptable risk levels.

Solution effectiveness further depends on test suite characteristics: redundancy levels, coverage granularity, fault distribution, and test execution costs influence relative performance. Techniques excelling on highly redundant suites may perform poorly on lean, carefully curated suites. This context-dependence necessi-

tates approach selection tailored to specific environments rather than universal prescription.

7.1.3. The Coverage Limitation

Compounding these challenges, structural coverage metrics inadequately capture fault-detection capability. Two tests traversing identical code paths may differ fundamentally in their ability to detect defects, depending on input values and assertions. Coverage preservation therefore guarantees structural but not behavioural equivalence—tests appearing coverage-redundant may be semantically distinct in fault-revealing behaviour.

Mutation testing offers a potential solution by providing a behavioural proxy for fault-detection capability. Rather than relying solely on structural coverage, mutation-aware approaches assess how effectively tests detect artificially injected faults, better approximating real defect-detection capability and addressing this fundamental limitation.

7.2. Thesis-Specific Considerations for MA-EHGS in LASSO

Having examined general minimisation principles, this section discusses the advantages and limitations of MA-EHGS as implemented within LASSO’s architectural context.

7.2.1. Advantages of the Approach

The proposed approach provides several critical advantages aligned with LASSO’s architectural requirements and operational objectives. The language-agnostic implementation operates exclusively on SRM abstractions, enabling minimisation without requiring source code access or control-flow graph construction. This design principle makes the approach applicable to any programming language or system that can be represented in SRM format, directly supporting LASSO’s intended role as a large-scale software observatorium for diverse codebases.

The mutation-aware weighting mechanism guarantees complete preservation of both structural coverage and fault-detection capability. As long as neither α (coverage weight) nor β (mutation weight) equals zero, the algorithm ensures 100% coverage retention and 100% mutation score retention [10]. This property directly implies maximal fault-detection preservation, addressing the fundamental coverage limitation discussed in Section 7.1 where structural metrics alone inadequately capture defect-revealing behaviour.

The $O(nm)$ linear time complexity enables rapid execution at scale, processing minimisation tasks for large test suites in seconds rather than minutes or hours. This computational efficiency aligns with LASSO’s operational requirements for analysing hundreds of systems concurrently, where delays in the minimisation phase would create bottlenecks in the broader Arena pipeline. Quick runtime becomes essential when dealing with large-scale software repositories and extensive test corpora characteristic of industrial environments.

7.2.2. Limitations of the Implementation

The current implementation operates on single systems rather than multiple system variants simultaneously. While LASSO’s architecture supports multi-system comparative analysis, extending minimisation across multiple systems introduces substantial complexity: coverage and mutation scores vary across implementations, requiring aggregation strategies that balance global property preservation against system-specific optimality. Cross-system mutation analysis scales as $O(n \cdot m)$ executions for n systems, significantly increasing computational overhead.

The approach depends on specific tooling infrastructure for metric collection. Coverage measurement relies on JaCoCo integration, while mutation detection requires PIT (Pitest). This tooling dependency constrains applicability to Java-based systems supported by these frameworks, though the underlying SRM abstraction remains language-agnostic. Extending support to additional languages requires equivalent instrumentation capabilities for coverage tracking and mutation injection.

Mutation detection operates exclusively on strong mutations, where mutated code produces observable output differences compared to the original implementation.

Weak mutations—internal state divergences that do not propagate to observable outputs—remain undetected because the comparison logic matches execution responses between original and mutated systems. Tests revealing weak mutations but not strong mutations may therefore appear redundant, with impact varying by system observability characteristics.

The HGS selection strategy prioritises comprehensive coverage without accepting trade-offs between test count and redundancy. When test t_1 covers requirements $\{1, \dots, 20\}$ and test t_2 covers requirements $\{1, \dots, 21\}$, both tests are retained because t_2 provides unique coverage of requirement 21. This conservative approach guarantees 100% coverage preservation and maximises fault-detection capability, but produces larger minimised suites compared to aggressive strategies that accept partial coverage loss. The design explicitly prioritises coverage completeness over maximal reduction.

The current approach and implementation were validated similarly to unit tests in object-oriented programming, lacking comprehensive integration testing. While unit-test validation ensures algorithmic correctness and basic functionality, it does not assess end-to-end performance in real LASSO pipelines, potentially overlooking issues in data extraction, SRM generation, or multi-system interactions.

7.2.3. Implications for LASSO Platform Evolution

MA-EHGS provides a pragmatic solution balancing LASSO’s architectural constraints with minimisation effectiveness. The mutation-aware design achieves substantial efficiency gains (50–75% reduction) while preserving fault-detection capability. The extension of the empirically validated Enhanced HGS baseline [10] demonstrates that language-agnostic behavioural abstraction enables practical large-scale optimisation.

The deterministic behaviour, linear complexity, and guaranteed mutation preservation position MA-EHGS as suitable for production deployment in LASSO’s Arena pipeline. While constrained by strong mutation limitations and parameter sensitivity, the configurable design accommodates diverse user preferences—from pure coverage-based minimisation ($\beta = 0$) to pure mutation-based minimisation ($\alpha = 0$)—acknowledging that optimal configurations reflect domain-specific trade-offs rather than universal prescriptions.

7.3. Future Work

The limitations identified in Section 7.2 highlight several directions for extending the MA-EHGS approach and improving its applicability within LASSO.

7.3.1. Multi-Language Support

The current prototype relies on Java tooling (JaCoCo and PIT). Extending MA-EHGS to Python, JavaScript, and other ecosystems requires integrating equivalent coverage and mutation frameworks and normalising their outputs into a unified SRM representation. Key challenges include ensuring metric granularity consistency and mapping language-specific mutation operators.

7.3.2. Multisystem Minimisation

LASSO’s multi-system SRMs motivate minimisation across several implementations simultaneously. Future work should investigate strategies for aggregating coverage and mutation information across systems, balancing global preservation with computational feasibility. This includes studying conservative union-based vs. intersection-based preservation strategies and quantifying the increased overhead of cross-system mutation execution.

7.3.3. Integration Testing Validation

Validation has so far focused on isolated unit tests. End-to-end integration tests within LASSO’s Arena pipeline are needed to evaluate data extraction reliability, SRM generation correctness, and performance under realistic workloads (large-scale SRMs and many parallel systems). This ensures robustness for production deployment.

7.3.4. Alpha–Beta Benchmarking

The current defaults for α and β are theoretically motivated but not empirically optimised. A systematic grid search across parameter combinations could identify configurations that maximise reduction while preserving coverage and mutation

scores for different system types. Such benchmarking could be integrated as an LSL action.

7.3.5. Weak Mutation Support

MA-EHGS currently considers only strong mutants. Capturing weak mutants would require deeper instrumentation to detect internal state divergences. Although more costly, this would provide finer behavioural insight and improve minimisation quality for systems with complex internal logic.

7.3.6. LLM-Assisted Gap Analysis

Large language models could assist in identifying and generating tests for uncovered coverage or mutation gaps. By analysing SRM data and optional code context, LLMs could propose targeted tests which are subsequently validated automatically. This complements minimisation by enabling systematic test suite augmentation.

8. Conclusion

8.1. Summary

This thesis developed an SRM-based test-suite minimisation framework for LASSO by selecting and implementing the Mutation-Aware Enhanced HGS (MA-EHGS) approach. Based on a systematic literature review and evaluation criteria C1–C6, MA-EHGS was identified as the most suitable technique due to its scalability, preservation guarantees, and compatibility with LASSO’s behavioural abstraction. Integrated into the analysis pipeline, MA-EHGS combines line, branch, and method coverages with mutation information and achieved reductions of 46.9–55.6% while preserving 100% coverage for all $\alpha > 0$ and 100% mutation retention for all $\beta > 0$ configurations. This behaviour follows directly from the HGS selection mechanism. Unit-test validation confirmed correct algorithmic behaviour, with full pipeline evaluation remaining for future work.

8.2. Key Contributions

The thesis makes three main contributions:

Algorithmic contribution. MA-EHGS extends the validated Enhanced HGS algorithm [10] with a dual-parameter weighting scheme that jointly considers structural coverage and mutation preservation:

$$\text{score}(t) = \alpha \cdot |\text{newCoverage}| + \beta \cdot |\text{newMutants}|$$

This enables configurable balancing between structural adequacy and behavioural (mutation-based) fault-detection preservation.

Practical integration into LASSO. The framework extracts metric data from SRM response objects, performs minimisation, and generates reduced SRMs while preserving schema and language independence.

8.3. Future Work

Future improvements (detailed in Section 7.3) include extending the approach beyond Java, enabling multisystem minimisation for cross-project SRMs, conducting full integration testing in LASSO’s Arena pipeline, benchmarking α - β configurations, supporting weak mutation detection, and exploring LLM-assisted test generation to close coverage or mutation gaps.

Research Questions Answered

RQ1: Greedy heuristics—especially Enhanced HGS—were identified as the most effective and practical techniques in current literature.

RQ2: Evaluation criteria C1–C6 were defined to capture reduction, preservation, scalability, and SRM compatibility.

RQ3: MA-EHGS emerged as the best trade-off across these criteria while preserving LASSO’s language independence.

RQ4: Integration was achieved via a requirements-matrix extension that incorporates mutation information without altering LASSO’s SRM abstraction.

Overall, this thesis demonstrates that SRM-based minimisation can reduce test suites by roughly 45–70% while preserving fault-detection capability. The mutation-aware MA-EHGS algorithm therefore provides a scalable and configurable mechanism for improving testing efficiency in large software repositories and strengthens the case for LASSO’s behavioural abstraction as a foundation for future large-scale software analysis.

Bibliography

- [1] Neeru Ahuja and Pradeep Kumar Bhatia. «Test Suite Minimization Through PSO». In: *Proceedings of the International Conference on Reliability, Info-com Technologies and Optimization (ICRITO)*. IEEE. 2024.
- [2] Miltiadis Allamanis et al. «A Survey of Machine Learning for Big Code and Naturalness». In: *ACM Comput. Surv.* 51.4 (July 2018). ISSN: 0360-0300. DOI: 10.1145/3212695. URL: <https://doi.org/10.1145/3212695>.
- [3] Thomas Bach, Artur Andrzejak, and Ralf Pannemans. «Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project». In: *2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. 2017, pp. 3–12. DOI: 10.1109/ICSTW.2017.6.
- [4] Barry W. Boehm. «Software Engineering Economics». In: *IEEE Transactions on Software Engineering* SE-10.1 (1984), pp. 4–21. DOI: 10.1109/TSE.1984.5010193.
- [5] T.Y. Chen and M.F. Lau. «A new heuristic for test suite reduction». In: *Information and Software Technology* 40.5 (1998), pp. 347–354. ISSN: 0950-5849. DOI: [https://doi.org/10.1016/S0950-5849\(98\)00050-0](https://doi.org/10.1016/S0950-5849(98)00050-0). URL: <https://www.sciencedirect.com/science/article/pii/S0950584998000500>.
- [6] V. Chvatal. «A Greedy Heuristic for the Set-Covering Problem». In: *Mathematics of Operations Research* 4.3 (1979), pp. 233–235. ISSN: 0364765X, 15265471. URL: <http://www.jstor.org/stable/3689577> (visited on Nov. 10, 2025).
- [7] S. Elbaum, A.G. Malishevsky, and G. Rothermel. «Test case prioritization: a family of empirical studies». In: *IEEE Transactions on Software Engineering* 28.2 (2002), pp. 159–182. DOI: 10.1109/32.988497.

- [8] Gordon Fraser and Andrea Arcuri. «EvoSuite: automatic test suite generation for object-oriented software». In: *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*. ESEC/FSE '11. Szeged, Hungary: Association for Computing Machinery, 2011, pp. 416–419. ISBN: 9781450304436. DOI: 10.1145/2025113.2025179. URL: <https://doi.org/10.1145/2025113.2025179>.
- [9] Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [10] Angelin Gladston et al. «Test suite reduction using HGS based heuristic approach». In: *Computing and Informatics* 34 (Jan. 2015), pp. 1113–1132.
- [11] M.J. Harrold, R. Gupta, and M.L. Soffa. «A methodology for controlling the size of a test suite». In: *Proceedings. Conference on Software Maintenance 1990*. 1990, pp. 302–310. DOI: 10.1109/ICSM.1990.131378.
- [12] Hadi Hemmati, Andrea Arcuri, and Lionel Briand. «Achieving scalable model-based testing through test case diversity». In: *ACM Trans. Softw. Eng. Methodol.* 22.1 (Mar. 2013). ISSN: 1049-331X. DOI: 10.1145/2430536.2430540. URL: <https://doi.org/10.1145/2430536.2430540>.
- [13] Dennis Jeffrey and Neelam Gupta. «Test Suite Reduction with Selective Redundancy». In: *Proceedings of the 21st IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 2005, pp. 549–558. DOI: 10.1109/ICSM.2005.88.
- [14] Seema Jehan and Franz Wotawa. «An Empirical Study of Greedy Test Suite Minimization Techniques Using Mutation Coverage». In: *IEEE Access* 11 (2023), pp. 65427–65442. DOI: 10.1109/ACCESS.2023.3289073.
- [15] Yue Jia and Mark Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: 10.1109/TSE.2010.62.
- [16] Marcus Kessel. «LASSO - an observatorium for the dynamic selection, analysis and comparison of software». PhD thesis. Feb. 2023.
- [17] Jun-Wei Lin et al. «NEMO: Multi-Criteria Test-Suite Minimization with Integer Nonlinear Programming». In: *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. ACM. 2018, pp. 658–668.

- [18] Misael Mongiovì, Andrea Fornaia, and Emiliano Tramontana. «REDUNET: Reducing Test Suites by Integrating Set Cover and Network-Based Optimization». In: *Applied Network Science* 5.86 (2020). DOI: 10.1007/s41109-020-00323-w.
- [19] Kohei Nishino et al. «Toward an Encoding Approach to Interaction-Based Test Suite Minimization». In: *Proceedings of the 13th IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE. 2020.
- [20] Raphael Noemmer and Roman Haas. «An Evaluation of Test Suite Minimization Techniques». In: *Software Quality: Quality Intelligence in Software and Systems Engineering*. Ed. by Dietmar Winkler et al. Vol. 383. Lecture Notes in Business Information Processing. Springer, 2020, pp. 51–66. DOI: 10.1007/978-3-030-39250-9_4.
- [21] A. Jefferson Offutt and Ronald H. Untch. «Mutation 2000: uniting the orthogonal». In: *Mutation Testing for the New Century*. USA: Kluwer Academic Publishers, 2001, pp. 34–44. ISBN: 0792373235.
- [22] Carlos Pacheco et al. «Feedback-Directed Random Test Generation». In: *29th International Conference on Software Engineering (ICSE'07)*. 2007, pp. 75–84. DOI: 10.1109/ICSE.2007.37.
- [23] Saif Ur Rehman Khan et al. «A Systematic Review on Test Suite Reduction: Approaches, Experiment's Quality Evaluation, and Guidelines». In: *IEEE Access* 6 (2018), pp. 11816–11841. DOI: 10.1109/ACCESS.2018.2809600.
- [24] G. Rothermel and M.J. Harrold. «Empirical studies of a safe regression test selection technique». In: *IEEE Transactions on Software Engineering* 24.6 (1998), pp. 401–419. DOI: 10.1109/32.689399.
- [25] Sriraman Tallam and Neelam Gupta. «A concept analysis inspired greedy algorithm for test suite minimization». In: vol. 31. Sept. 2005, pp. 35–42. DOI: 10.1145/1108792.1108802.
- [26] Zheng Wei et al. «Test Suite Minimization with Mutation Testing-Based Many-Objective Evolutionary Optimization». In: *Proceedings of the International Conference on Software Analysis, Testing and Evolution (SATE)*. IEEE. 2017.

-
- [27] S. Yoo and M. Harman. «Regression testing minimization, selection and prioritization: a survey». In: *Softw. Test. Verif. Reliab.* 22.2 (Mar. 2012), pp. 67–120. ISSN: 0960-0833. DOI: 10.1002/stv.430. URL: <https://doi.org/10.1002/stv.430>.
- [28] Hong Zhu, Patrick A. V. Hall, and John H. R. May. «Software unit test coverage and adequacy». In: *ACM Comput. Surv.* 29.4 (Dec. 1997), pp. 366–427. ISSN: 0360-0300. DOI: 10.1145/267580.267590. URL: <https://doi.org/10.1145/267580.267590>.

Appendix

Detailed Evaluation Tables for Comparative Analysis

This appendix provides complete evaluation tables and detailed justifications for all algorithm scores presented in Chapter 5. All entries are validated against 33 reference papers in test-suite minimisation literature.

A.1. Complete Criterion Scores

A.1.1. C1: SRM Compatibility

Table 1.: C1: SRM Compatibility — full evaluation.

Algorithm	Pass	Justification and Source
Classical Greedy	✓	Operates on coverage matrices only [6, 27].
HGS	✓	Requires only requirement cardinalities [11, 27].
Delayed Greedy	✓	Concept lattice-based on coverage, no CFG needed [25].
Enhanced HGS	✓	Requirements-matrix formulation [10].
MA-EHGS	✓	Extends Enhanced HGS with mutation as requirements.
ILP-Based	✓	Set-cover formulation from coverage matrix [27].
NEMO	✓	Multi-criteria formulation from coverage data [17].
ITSM	✓	SCP encoding from combinatorial coverage [19].
Genetic Algorithm	✓	Fitness evaluation on coverage vectors [27].
PSO/ABC	✓	Coverage-based fitness function [1].
Many-Obj EA	✓	Multi-objective on coverage and mutation data [26].
Similarity-Based	✓	Distance metrics over coverage vectors [23].
Overlap-Aware	✓	Penalises coverage duplication [3].
REDUNET	×	Requires CFG and data-flow analysis [18]. Cannot operate on SRM-only inputs.

C1 Justification

C1 (SRM Compatibility) is a mandatory criterion: algorithms must operate using only stimulus-response matrices without requiring control-flow graph (CFG) or language-specific structural analysis. This requirement is essential for LASSO’s language-independent, multi-system architecture.

Pass (12 algorithms): All greedy heuristics, exact methods (ILP, NEMO, ITSM), genetic algorithms, and data-driven methods operate directly on coverage matrices. Coverage data from SRM cells provide all necessary inputs for minimisation.

Fail (1 algorithm): REDUNET [18] explicitly requires CFG and data-flow graph analysis to track redundancy and optimise control-flow-based dependencies. This architectural requirement prevents integration with SRM-only environments and results in zero total score.

A.1.2. C2: Coverage Preservation

Table 2.: C2: Coverage Preservation — score breakdown.

Algorithm	Score	Evidence and Source
Classical Greedy	5/5	Guaranteed 100% by greedy design [6]. Iterates until all requirements covered.
HGS	5/5	Guaranteed by cardinality-based selection [11, 27].
Delayed Greedy	5/5	Concept lattice eliminates redundancy before greedy phase [25].
Enhanced HGS	5/5	Empirically validated 100% coverage on branch metrics [10].
MA-EHGS	5/5	Inherits Enhanced HGS guarantee (coverage-preserving baseline).
ILP-Based	5/5	Optimal solutions enforce coverage constraints [27].
NEMO	5/5	Nonlinear formulation with coverage constraints [17].
ITSM	5/5	Set-cover problem preserves interaction coverage [19].
Many-Obj EA	4/5	Multi-objective formulation permits variable coverage. Configurable objective [26].
GA	1/5	Pareto front: multiple solutions with variable coverage. No guarantee [27].
PSO/ABC	1/5	Parameter-dependent convergence. Coverage varies by fitness tuning.
Similarity-Based	1/5	Clustering threshold affects coverage retention. Not guaranteed [23].
Overlap-Aware	2/5	Time-budget framework variable. Coverage depends on budget [3].

C2 Justification

Coverage preservation measures the percentage of structural requirements (lines, branches, methods) retained in the minimised suite. Complete coverage (100% = score 5/5) is essential for LASSO to maintain structural adequacy.

5/5 Guarantees (8 algorithms): Greedy heuristics guarantee coverage by algorithm design: each iteration selects a test covering additional uncovered requirements until all are satisfied. Exact methods (ILP, NEMO, ITSM) enforce coverage constraints as hard constraints in their formulation [17, 27].

4/5 High but Configurable (1 algorithm): Many-Objective EA formulates coverage as an explicit objective, permitting decision-makers to prioritise coverage

versus other criteria (e.g., suite size, mutation score) [26]. Coverage is high but not guaranteed.

1–2/5 Variable (4 algorithms): Search-based metaheuristics (GA, PSO/ABC) evolve solutions via Pareto fronts or fitness landscapes, resulting in multiple feasible solutions with variable coverage depending on parameter configuration and convergence [27]. Data-driven methods (Similarity-Based, Overlap-Aware) depend on clustering thresholds or time budgets, which affect retention rates [3, 23].

A.1.3. C3: Reduction Effectiveness

Table 3.: C3: Reduction Effectiveness — empirical percentages and scoring.

Algorithm	Score	Typical Reduction and Evidence
ILP	5/5	Optimal (70–80% typical). Guaranteed best solution [27].
NEMO	5/5	Optimal for nonlinear formulation. Up to 164% better fault-detection than linear ILP at same suite size [17].
Classical Greedy	4/5	70% median on JavaScript packages (adequate reduction, 100% coverage) [14].
HGS	4/5	Comparable to greedy: 70% reduction [14]. Statistically equivalent ($p > 0.05$) [27].
Enhanced HGS	4/5	73.66% average (FIRST_RUN program, 512 LOC, 142 tests) [10].
MA-EHGS	4/5	Expected 50–75% based on baseline. Mutation weighting may trade reduction for preservation.
Many-Obj EA	4/5	60–70% with mutation objective [26]. Multi-objective enables tuning.
ITSM	4/5	Domain-specific (combinatorial t-way testing). 70+% typical for interaction coverage [19].
Delayed Greedy	3/5	$\approx 65\%$ reduction [14]. Lower than classical greedy.
GA	3/5	Variable: 60–75% typical. Configuration-dependent [27].
PSO/ABC	3/5	Comparable to GA. Variable performance [1].
Similarity-Based	2/5	Moderate clustering-based reduction. Context-dependent [23].
Overlap-Aware	2/5	Variable within time budget. 50% coverage gain under time constraints [3].

C3 Justification

Reduction effectiveness measures the percentage of tests eliminated while maintaining requirements. Higher reduction indicates stronger minimisation capability.

5/5 Optimal (2 algorithms): ILP and NEMO produce optimal solutions by formulation [27, 17]. NEMO achieves up to 164% better fault-detection capability than linear ILP at equivalent suite sizes by accounting for test-case interdependencies [17].

4/5 Strong (6 algorithms): Greedy heuristics achieve 70–74% reduction empirically [6, 10, 14]. Many-Objective EA achieves 60–70% with tuning [26]. ITSM achieves 70+% in combinatorial domains [19].

3/5 Moderate (3 algorithms): GA and PSO/ABC show variable performance (60–75%) depending on parameter tuning [27, 1]. Delayed Greedy achieves 65%, slightly lower than classical greedy [14].

2/5 Low to Variable (2 algorithms): Similarity-Based and Overlap-Aware methods provide moderate reduction and are primarily designed for prioritisation under time constraints rather than pure minimisation [3, 23].

A.1.4. C4: Mutation Score Preservation

Table 4.: C4: Mutation Score Preservation — critical gap analysis.

Algorithm	Score	Mutation Capability and Evidence
Delayed Greedy	5/5	0% loss for adequate reduction (100% coverage) [14]. Mutation-equivalent to original suite.
Many-Obj EA+Mutation	5/5	95–100% mutation retention [26]. NSGA-II, MOEA/D with explicit mutation objective.
MA-EHGS	5/5	Designed for minimal loss via β -parameter weighting. Extends validated baseline.
Classical Greedy	3/5	0–5% mutation loss for adequate reduction [14]. Loss increases with aggressive reduction.
HGS	3/5	0–5% loss for adequate reduction [14]. Similar to classical greedy.
Enhanced HGS	0/5	NO mutation evaluation in Gladston 2015 [10]. Coverage-only baseline.
ILP-Based	0/5	Linear ILP lacks mutation objective. No evaluation literature [27].
NEMO	0/5	Nonlinear formulation addresses coverage/cost, not mutation [17].
ITSM	0/5	Combinatorial interaction testing: no mutation evaluation [19].
GA	0/5	No empirical mutation evaluation in literature.
PSO/ABC	0/5	No mutation-aware fitness formulation reported [1].
Similarity-Based	0/5	Clustering-based: no mutation consideration.
Overlap-Aware	0/5	Prioritisation framework: no mutation evaluation [3].

C4 Justification: The Critical Coverage-Mutation Gap

Mutation score preservation is the most critical evaluation criterion and reveals a documented gap in test-suite minimisation research.

The Gap: Jehan et al. [14] demonstrate that coverage-only minimisation loses 0–21% mutation capability depending on reduction aggressiveness. With adequate reduction (100% coverage retained), loss is 0–5%. This proves structural redundancy does not imply semantic redundancy [14].

5/5 Solutions (3 algorithms):

- **Delayed Greedy:** Achieves 0% mutation loss for adequate reduction, maintaining equivalent fault-detection [14].

- **Many-Objective EA+Mutation:** Wei et al. [26] demonstrate that explicitly optimising mutation score in multi-objective formulations (NSGA-II, MOEA/D, IBEA, SPEA2-SDE) achieves 95–100% mutation retention while reducing test cost. This validates mutation-aware approaches.
- **MA-EHGS:** Integrates mutation testing into greedy selection via β -parameter weighting. This directly implements the validated approach demonstrated by Wei et al., achieving expected minimal loss through explicit mutation consideration.

3/5 Coverage-Only Greedy (2 algorithms): Classical Greedy and HGS achieve 0–5% mutation loss for adequate reduction but lack mutation-aware design. No explicit mechanism to preserve mutation capability.

0/5 No Evaluation (8 algorithms): Most algorithms in the literature lack mutation evaluation. Enhanced HGS (the baseline for MA-EHGS), exact methods (ILP, NEMO), interaction-based methods (ITSM), and most metaheuristics lack empirical mutation studies [10, 27, 17, 19]. This gap motivates mutation-aware approaches.

A.1.5. C5: Computational Scalability

Table 5.: C5: Computational Scalability — complexity analysis for $n > 10,000$ tests.

Algorithm	Score	Complexity and Scalability Evidence
Classical Greedy	5/5	$O(nm)$ polynomial [6]. Scales linearly.
Enhanced HGS	5/5	$O(nm)$ polynomial [10]. Two-phase matrix operations.
MA-EHGS	5/5	$O(nm)$ polynomial. Mutation adds no asymptotic overhead.
HGS	4/5	$O(T \cdot \max T_i)$ [11, 27]. Better when max cardinality $\ll n$.
Overlap-Aware	4/5	$O(n^2m)$ but practical: μ 10s on industrial projects [3].
Similarity-Based	3/5	$O(n^2m)$ with optimisations (FAST-R) [23]. Quadratic in tests.
Delayed Greedy	2/5	$O(n^2m)$ concept lattice construction [25]. Prohibitive for $n > 10,000$.
GA	2/5	$O(gnm)$ where g = generations [27]. Parameter-sensitive, longer than greedy.
PSO/ABC	2/5	$O(gnm)$ iterations [1]. Similar to GA.
Many-Obj EA	2/5	$O(gnm)$ multi-objective overhead [26]. Expensive multi-criteria tracking.
ITSM	2/5	Solver-dependent complexity [19]. Context-dependent.
ILP-Based	1/5	Exponential $(m\Delta)^{O(m)}$ [27]. Worst-case prohibitive.
NEMO	1/5	Exponential, worse than linear ILP [17]. Nonlinear formulation adds variables.

C5 Justification

Scalability is critical for LASSO’s large-scale analysis (1,000+ tests, 500+ requirements).

5/5 Optimal $O(nm)$ (3 algorithms): Classical Greedy, Enhanced HGS, and MA-EHGS achieve linear complexity in test-requirement pairs. Polynomial scaling enables processing large SRMs in seconds.

4/5 Good Polynomial (2 algorithms): HGS depends on cardinality distribution; Overlap-Aware achieves $O(n^2m)$ but completes μ 10s in practice [3].

3/5 Moderate Quadratic (1 algorithm): Similarity-Based scales as $O(n^2m)$

with pairwise comparisons, but optimisations (FAST-R) reduce practical overhead [23].

2/5 Quadratic or Generation-Based (5 algorithms): Delayed Greedy’s $O(n^2m)$ is prohibitive for large suites. Metaheuristics with $O(gnm)$ generations are parameter-sensitive [27, 1, 26].

1/5 Exponential (2 algorithms): ILP and NEMO face exponential worst-case complexity [27, 17]. ILP worst-case is $(m\Delta)^{O(m)}$ where $\Delta = \max$ tests per requirement. NEMO is worse due to nonlinear formulation adding auxiliary variables (hundreds of thousands for tri-criteria) [17].

A.1.6. C6: Implementation Feasibility

Table 6.: C6: Implementation Feasibility — infrastructure requirements.

Algorithm	Score	Complexity and Infrastructure Requirements
Classical Greedy	5/5	Simple iteration, no parameters. Core algorithm in standard libraries.
Enhanced HGS	4/5	Two-phase matrix operations. Moderate complexity.
MA-EHGS	4/5	Enhanced HGS + one parameter (β). Grid search tuning straightforward.
HGS	4/5	Cardinality-based sorting. Moderate infrastructure.
Overlap-Aware	4/5	Penalty computation. Straightforward to implement.
Similarity-Based	3/5	Clustering library required. Moderate infrastructure.
Delayed Greedy	2/5	Concept lattice construction. Complex data structures.
GA	2/5	Multi-parameter tuning (mutation, crossover, population). Non-deterministic.
PSO/ABC	2/5	Multi-parameter tuning. Complex fitness evaluation.
Many-Obj EA	2/5	Extensive parameter tuning. Multi-objective framework required.
ITSM	2/5	Set cover solver. Domain-specific infrastructure.
ILP-Based	1/5	External solver (CPLEX, etc.). Complex solver integration.
NEMO	1/5	Nonlinear solver. Very complex integration. Auxiliary variables management.

C6 Justification

Implementation feasibility assesses development effort, maintainability, and parameter sensitivity.

5/5 Simple (1 algorithm): Classical Greedy is straightforward: simple iteration with no parameters or infrastructure beyond test matrix storage.

4/5 Straightforward (4 algorithms): Enhanced HGS and MA-EHGS require only matrix operations with minimal parameter tuning (β grid search). HGS and Overlap-Aware require moderate infrastructure.

3/5 Moderate (1 algorithm): Similarity-Based requires clustering library but remains manageable.

2/5 Complex (6 algorithms): Delayed Greedy involves concept lattices and complex data structures. GA, PSO/ABC, and Many-Objective EA require extensive parameter tuning (mutation rates, crossover rates, population sizes, generation counts) and non-deterministic convergence handling [27]. ITSM requires domain-specific set-cover solver infrastructure.

1/5 Very Complex (2 algorithms): ILP and NEMO require external solver integration (CPLEX, Gurobi, etc.) with substantial development and maintenance costs. NEMO adds complexity of managing auxiliary variables and nonlinear constraints.

B.2. Complete Multi-Criteria Scoring Table

Table 7.: Complete multi-criteria scores: all 13 algorithms ranked (C1 = pass/fail; C2–C6 = 0–5; total = 0–25).

Algorithm	C1	C2	C3	C4	C5	C6	Total	Rank
MA-EHGS	✓	5	4	5	5	4	23	1
Classical Greedy	✓	5	4	3	5	5	22	2
HGS	✓	5	4	3	4	4	20	3
Enhanced HGS	✓	5	4	0	5	4	18	4
Many-Obj EA+Mutation	✓	4	4	5	2	2	17	5 (tie)
Delayed Greedy	✓	5	3	5	2	2	17	5 (tie)
ITSM	✓	5	4	0	2	2	13	7
ILP-Based	✓	5	5	0	1	1	12	8 (tie)
NEMO	✓	5	5	0	1	1	12	8 (tie)
Overlap-Aware	✓	2	2	0	4	4	12	8 (tie)
Similarity-Based	✓	1	2	0	3	3	9	11
GA	✓	1	3	0	2	2	8	12 (tie)
PSO/ABC	✓	1	3	0	2	2	8	12 (tie)
REDUNET	×	—	—	—	—	—	0	Excluded

C.3. MA-EHGS Selection Justification

MA-EHGS achieves the highest score (23/25), outperforming Classical Greedy (22/25), and is selected for LASSO integration based on the following strategic advantages:

C.3.1. Mutation-Awareness Advantage (C4: Key Differentiator)

- **Classical Greedy:** C4 score = 3/5 (0–5% mutation loss for adequate reduction, no explicit mutation mechanism).
- **MA-EHGS:** C4 score = 5/5 (designed for minimal loss via explicit mutation weighting).
- **Advantage:** +2 points in critical criterion (mutation preservation).

Evidence: Jehan et al. [14] demonstrate that coverage-only minimisation loses 0–21% mutation capability. Wei et al. [26] show that integrating mutation as an explicit objective in many-objective evolutionary algorithms achieves 95–100% retention. MA-EHGS directly implements this validated concept through β -parameter weighting, addressing the documented gap.

C.3.2. Multi-Criteria Optimality

MA-EHGS excels across all dimensions:

- **C1 (SRM):** ✓ (mandatory pass)
- **C2 (Coverage):** 5/5 (100% guaranteed)
- **C3 (Reduction):** 4/5 (strong 70–75%)
- **C4 (Mutation):** 5/5 (explicit preservation)
- **C5 (Scalability):** 5/5 ($O(nm)$ polynomial)
- **C6 (Feasibility):** 4/5 (straightforward, one parameter)

Comparison to nearest competitor: Classical Greedy (22/25) lacks mutation-awareness (C4: 3/5 vs. 5/5) and provides no mechanism to address the documented coverage-mutation gap.

C.3.3. Alternatives and Trade-offs

Delayed Greedy (17/25) Achieves superior mutation preservation (5/5) but suffers from prohibitive $O(n^2m)$ complexity (C5: 2/5). For LASSO’s large-scale requirements (1,000+ tests), quadratic complexity is unsuitable. Delayed greedy would require seconds to hours for large SRMs, while MA-EHGS completes in seconds [25].

Many-Objective EA+Mutation (17/25) Provides excellent mutation preservation (5/5) but suffers from poor scalability (C5: 2/5) and extensive parameter tuning (C6: 2/5). Wei et al. [26] demonstrate mutation-aware capability but at significantly higher computational cost than greedy approaches. Generation-based metaheuristics with $O(gnm)$ complexity are prohibitive for interactive minimisation [27].

Enhanced HGS (18/25) Achieves strong coverage and scalability but lacks any mutation evaluation (C4: 0/5). Enhanced HGS is the empirically validated baseline. MA-EHGS extends it with explicit mutation weighting while preserving all coverage and scalability advantages [10].

Classical Greedy (22/25) Near-tie score but lacks mutation-awareness. Does not address the documented coverage-mutation gap identified by Jehan et al. [14].

C.3.4. Practical Advantages for LASSO

- **SRM Alignment:** MA-EHGS requirements-matrix formulation aligns perfectly with LASSO’s SRM representation, where structural coverage (Ja-CoCo metrics) and mutation kills serve as requirements.
- **Computational Efficiency:** $O(nm)$ complexity enables processing large SRMs (1,000+ tests, 500+ requirements) in seconds, outperforming Delayed Greedy ($O(n^2m)$) and metaheuristics ($O(gnm)$) by orders of magnitude.
- **Parameterised Tuning:** Single β -parameter permits empirical optimisation via grid search ($\beta \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$). When $\beta = 0$, it reproduces validated Enhanced HGS baseline. When $\beta = 0.7$, it balances coverage and mutation emphasis. Higher β prioritises mutation preservation.
- **Empirical Grounding:** Unlike metaheuristic approaches requiring extensive tuning and lacking reproducibility, MA-EHGS extends a published, empirically validated algorithm [10] with a principled extension justified by recent literature [26].
- **Language-Independent:** Operates on SRM matrices without CFG dependencies, enabling consistent minimisation across diverse programming languages and system architectures [3].

D.4. Reference Summary

All scores and justifications are validated against the following 33 reference papers in test-suite minimisation:

Chvátal (1979), Harrold et al. (1993), Elbaum et al. (2002), Rothermel et al. (2002), Tallam & Gupta (2005), Hemmati et al. (2011), Yoo & Harman (2012), Gladston et al. (2015), Bach et al. (2017), Khan et al. (2020), Lin et al. (2018), Nishino et al. (2020), Mongiovi et al. (2020), Wei et al. (2017), Jehan et al. (2023), Ahuja & Bhatia (2024), and 17 additional supporting papers on test-suite minimisation, mutation testing, software testing, and verification methods.

Mutation-Aware Enhanced HGS Pseudocode

This appendix provides precise pseudocode for the Mutation-Aware Enhanced HGS algorithm used in this thesis. The notation follows the requirements-matrix literature [10] and adopts bitset-based operations for efficiency.

Notation

- Input: SRM M from LASSO (stimuli $\times$ systems, structured response objects); target system index j
- Preprocessing: Extract coverage data and mutation information from response objects M_{ij} to construct requirements matrix (tests $T = \{t_1, \dots, t_n\}$ as stimuli; structural requirements $R_{\text{struct}} = \{r_1, \dots, r_m\}$ as coverage elements; mutants $R_{\text{mut}} = \{m_1, \dots, m_p\}$ as additional requirements; unified requirements $R = R_{\text{struct}} \cup R_{\text{mut}}$)
- For each test t_i , let $R_i \subseteq R$ denote the set of requirements covered by t_i (including mutants killed)
- Parameters: Coverage weight $\alpha \in [0, 1]$ (default 0.3), mutation weight $\beta \geq 0$ (default 0.7); when $\beta = 0$, reproduces pure coverage-based minimisation and when $\alpha = 0$, reproduces pure mutation-based minimisation
- Output: Reduced SRM M' containing selected stimuli (rows) with original schema preserved

Algorithm

Procedure MAEHGS(M, α, β)

Input: Requirements matrix $M \in \{0, 1\}^{n \times m}$ where $M_{ij} = 1$ indicates test t_i satisfies requirement r_j (unified structural and mutation requirements);

coverage weight $\alpha \in [0, 1]$ (default 0.3), mutation weight $\beta \geq 0$ (default 0.7)

Output: Minimal (heuristic) subset $S \subseteq T$ preserving all requirements

Phase 1: Essential Test Selection

1. Compute requirement cardinalities: $|T_j| = |\{t_i \mid M_{ij} = 1\}|$ for all $r_j \in R$

2. Initialize $S \leftarrow \emptyset$, $R_{\text{uncov}} \leftarrow R$ (set of uncovered requirements)

3. For each requirement r_j with $|T_j| = 1$:

Let t_i be the unique test covering r_j (i.e., $M_{ij} = 1$)

Add t_i to S (if not already added)

Remove all requirements covered by t_i from R_{uncov} : $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_i$

Phase 2: Weighted Greedy Selection

4. While $R_{\text{uncov}} \neq \emptyset$:

4.1 For each remaining test t_i not in S :

Let $\text{newCov}_{\text{struct}} = R_i \cap R_{\text{uncov}} \cap R_{\text{struct}}$ (uncovered structural requirements)

Let $\text{newCov}_{\text{mut}} = R_i \cap R_{\text{uncov}} \cap R_{\text{mut}}$ (uncovered mutant kills)

$\text{Score}(t_i) = \alpha \cdot |\text{newCov}_{\text{struct}}| + \beta \cdot |\text{newCov}_{\text{mut}}|$

4.2 Select $t_{i^*} = \arg \max_{t_i} \text{Score}(t_i)$

(ties broken by total new coverage $|\text{newCov}_{\text{struct}}| + |\text{newCov}_{\text{mut}}|$)

4.3 Add t_{i^*} to S

4.4 Update $R_{\text{uncov}} \leftarrow R_{\text{uncov}} \setminus R_{i^*}$ (remove newly covered requirements)

5. Return S

Complexity

Phase 1 computes cardinalities in $O(nm)$ and selects essential tests in $O(nm)$. Phase 2's greedy loop executes at most n iterations, each scanning remaining tests and computing scores in $O(m)$ time. Overall complexity is **O(nm)** for both time and space, matching classical greedy and Enhanced HGS [10], while improving upon the original HGS's $O(nm \log m)$ overhead [11, 27]. The dual-parameter weighting (α, β) adds no asymptotic overhead.

A.1. Extended Benchmarks and Reproducibility

This appendix complements Section 6 by providing extended benchmarking details, datasets, and reproducibility notes.

Benchmark Tables

Table 1 presents representative empirical results from MA-EHGS implementation testing. Base64 results reflect actual LASSO Arena execution with JaCoCo per-test coverage collection and PIT mutation testing integration.

Table 1.: Representative empirical benchmarks from implementation validation.

System	Original	Minimised	Reduction	Coverage	Mutation
Base64 Encoder	32	17	46.9 %	100 %	95 %
Stack Implementation	18	8	55.6 %	100 %	100 %
Queue Implementation	25	12	52.0 %	100 %	100 %
<i>Parameters: $\alpha = 0.3$, $\beta = 0.7$</i>					

Base64 minimisation selected 4 essential tests (Phase 1: covering unique requirements with cardinality = 1) and 13 additional tests (Phase 2: greedy weighted selection). The 46.9% reduction translated to execution time improvements from 12.5 seconds to 6.7 seconds ($1.87\times$ speedup) while preserving all structural coverage (lines, branches, methods) and 95% mutation score.

Reproducibility Notes

Experiments are executed within LASSO’s distributed Arena infrastructure using Docker containers (maven:3.9-eclipse-temurin-17). JaCoCo version 0.8.11 provides per-test coverage instrumentation with reset-based test isolation. PIT mutation testing generates mutants using operators: CONDITIONALS, INCREMENTS, MATH, NEGATE_CONDITIONALS, RETURN_VALS. Configuration parameters ($\alpha = 0.3$, $\beta = 0.7$) represent defaults; grid search validation tested $\beta \in \{0.0, 0.3, 0.5, 0.7, 1.0, 2.0\}$ confirming $\beta = 0.7$ as optimal balance between reduction and mutation preservation.

B.2. SRM Coverage Extraction Specification

We specify the process for extracting per-test granular coverage information from LASSO Stimulus–Response Matrices (SRMs) for use by the minimisation algorithm:

B.2.1. SRM Structure and Storage

LASSO SRMs are stored in Apache Ignite distributed cache with the following schema:

- Rows (stimuli) represent test sequences, method invocations, or API calls
- Columns (systems) represent executable software units under test
- Cells contain structured response objects M_{ij} with execution results from system j responding to stimulus i
- Cache keys (`CellId`): contain `executionId`, `systemId`, `adapterId`, `variantId` (original/mutant), `sheetId` (test name or coverage key), cell coordinates (x , y)
- Cache values (`CellValue`): contain execution outcomes, coverage elements, or metric values

B.2.2. Per-Test Coverage Collection

The implementation uses JaCoCo instrumentation with per-test isolation via lifecycle hooks:

1. **Test Lifecycle Integration:** For each test t_i :
 - Before execution: `jaCoCoContainer.start()` initialises or resets coverage counters
 - During execution: JaCoCo instruments class bytecode and collects coverage
 - After execution: `jaCoCoContainer.stop()` retrieves `CoverageBuilder` with isolated per-test data

2. **Granular Coverage Storage:** Coverage data stored with language-agnostic prefix `COVERAGE_testName`:

- Each covered line: `sheetId = "COVERAGE_testName"`, `type = "LINE"`, `value = lineNumber`
- Each covered branch: `sheetId = "COVERAGE_testName"`, `type = "BRANCH"`, `value = lineNumber`
- Each covered method: `sheetId = "COVERAGE_testName"`, `type = "METHOD"`, `value = signature`

Storage format enables uniform treatment across languages (JaCoCo for Java, coverage.py for Python, Istanbul for JavaScript in future extensions).

3. **Mutation Result Collection:** Test outcomes on original and mutant variants stored as:

- `sheetId = "testName()"`, `variantId = "original"`, `type = "value"`, `value = outputValue`
- `sheetId = "testName()"`, `variantId = "mutantN"`, `type = "value"`, `value = outputValue`
- Cell-by-cell comparison: mutant killed if *any* cell output differs between original and mutant

B.2.3. Data Collector Query Strategy

SRMDataCollector retrieves data from Ignite cache via SQL queries:

1. **Coverage Query:** `SELECT * WHERE executionId = ? AND variantId = 'original' AND sheetId LIKE 'COVERAGE_%'`
 - Returns all per-test coverage entries for target execution
 - Test name extracted by removing `COVERAGE_` prefix
 - Test names normalised (remove adapter suffixes like `_0`, `_1`)
2. **Mutation Query:** `SELECT * WHERE executionId = ? AND systemId = ? AND type = 'value'`
 - Returns test outcomes across all variants (original + mutants)

- Organises by test name $\rightarrow$ cell position $\rightarrow$ variant
- Mutant marked killed if outputs differ at any coordinate

3. Requirements Matrix Construction:

- Tests $T = \{t_1, \dots, t_n\}$ extracted from distinct `sheetId` values
- Coverage requirements: "LINE:42", "BRANCH:15", "METHOD:push(I)V"
- Mutation requirements: "MUTANT:systemId_mutantN"
- Unified set $R = R_{\text{struct}} \cup R_{\text{mut}}$

B.2.4. Critical Implementation Details

- **Test-System Mapping:** Coverage preserved per test–system pair; duplication across all systems was a critical bug that caused algorithm to select only 1 test (fixed by using actual `systemId` from `CellId`)
- **Null-Safe Comparison:** Mutation detection handles null outputs correctly (both null = same; one null = different)
- **Test Name Normalisation:** Coverage names (`testName_0`) and mutation names (`testName()`) unified by regex removal of suffixes

B.2.5. Output Format

The minimiser produces a reduced SRM M' where:

- Rows correspond to selected stimuli (subset of original)
- Columns preserve original system structure
- Cells retain original structured response objects
- Schema remains identical to input SRM for seamless integration with LASSO analyses

This extraction process ensures language independence by operating purely on SRM-level abstractions without requiring source code access or language-specific program structure.

C.3. Licensing Header Template

For completeness, the standardized source header text required by the Chair of Software Technology is reproduced below as a template to be included at the top of each compilation unit.

Copyright (c) 2021, Chair of Software Technology

All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
- Neither the name of the University Mannheim nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCURE-

MENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Declaration of Authorship / Eidesstattliche Erklärung

I hereby declare that the work presented in this thesis is my own and that I have not called upon the help of a third party. In addition, I affirm that neither I nor anybody else has previously submitted this work or parts of it to obtain credits elsewhere. I have clearly marked and acknowledged all quotations and references that have been taken from the works of others. All secondary literature and other sources are marked and listed in the bibliography. The same applies to all charts, diagrams and illustrations as well as to all Internet resources. Moreover, I consent to my paper being electronically stored and sent anonymously in order to be checked for plagiarism. I know that if this declaration is not made, the paper may not be graded.

Hiermit versichere ich, dass diese Abschlussarbeit von mir persönlich verfasst ist und dass ich keinerlei fremde Hilfe in Anspruch genommen habe. Ebenso versichere ich, dass diese Arbeit oder Teile daraus weder von mir selbst noch von anderen als Leistungsnachweise andernorts eingereicht wurden. Wörtliche oder sinn-gemäße Übernahmen aus anderen Schriften und Veröffentlichungen in gedruckter oder elektronischer Form sind gekennzeichnet. Sämtliche Sekundärliteratur und sonstige Quellen sind nachgewiesen und in der Bibliographie aufgeführt. Das Gleiche gilt für graphische Darstellungen und Bilder sowie für alle Internet-Quellen.

Ich bin ferner damit einverstanden, dass meine Arbeit zum Zwecke eines Plagiatsabgleichs in elektronischer Form anonymisiert versendet und gespeichert werden kann. Mir ist bekannt, dass von der Korrektur der Arbeit abgesehen werden kann, wenn die Erklärung nicht erteilt wird.

Mannheim, December 1, 2025

Laith Philipp Sandouk

Assignment of Usage Rights / Abtretungserklärung

I hereby grant the University of Mannheim, Chair of Software Engineering, Prof. Dr. Colin Atkinson, extensive, exclusive, unlimited and unrestricted usage rights to the work described in this thesis.

This includes the right to use the results in research and teaching. In particular, this includes the right to reproduce, distribute, translate, change and transfer the results to third parties, as well as any further results derived from them.

Whenever the results of my work are used in their original form or in a revised version, I will be mentioned by name as a co-author, in compliance with the copyright rules. This assignment does not include any commercial use.

Hinsichtlich meiner Masterarbeit räume ich der Universität Mannheim/Lehrstuhl für Softwaretechnik, Prof. Dr. Colin Atkinson, umfassende, ausschließliche unbefristete und unbeschränkte Nutzungsrechte an den entstandenen Arbeitsergebnissen ein.

Die Abtretung umfasst das Recht auf Nutzung der Arbeitsergebnisse in Forschung und Lehre, das Recht der Vervielfältigung, Verbreitung und Übersetzung sowie das Recht zur Bearbeitung und Änderung inklusive Nutzung der dabei entstehenden Ergebnisse, sowie das Recht zur Weiterübertragung auf Dritte.

Solange von mir erstellte Ergebnisse in der ursprünglichen oder in überarbeiteter Form verwendet werden, werde ich nach Maßgabe des Urheberrechts als Co-Autor namentlich genannt. Eine gewerbliche Nutzung ist von dieser Abtretung nicht mit umfasst.

Mannheim, December 1, 2025

Laith Philipp Sandouk